JEE ENTITIES AND COMPLEX CONTENT

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 | JAXB 2.3 | Maven 3.8+
Namespace: javax.ws.rs.* / javax.xml.bind.* (NOT jakarta.*)
Data format: XML — @Produces(APPLICATION_XML) / @Consumes(APPLICATION_XML)
Starter project: entities-lab-starter.zip
Labs: 4 Labs — approx. 3 hours total

Lab Overview & Prerequisites

Package Namespace & Data Format — Important

All code uses javax.ws.rs.* and javax.xml.bind.* (NOT jakarta.*) — correct for Java 11 / Tomcat 9.

All resource endpoints use @Produces(APPLICATION_XML) and @Consumes(APPLICATION_XML).
Domain model classes carry @XmlRootElement so Jersey/JAXB marshals them automatically.
Lab 1 CSV provider: text/csv (unchanged — this is a custom format, not JSON/XML).

Correct imports:

```
import javax.ws.rs.*;   import javax.ws.rs.core.*;   import javax.ws.rs.ext.*;
import javax.xml.bind.*;   import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.*;
```

Labs in This Workbook

Lab	Title	Covers	Section	Time
1	Custom Entity Provider — CSV	MessageBodyWriter, MessageBodyReader, isWriteable	Sec 1	45 min
2	ContextResolver<JAXBContext>	Shared JAXBContext, adapters, namespace	Sec 2	30 min
3	JAXB Annotations — XML Output	@XmlRootElement, @XmlAttribute, @XmlElementWrapper	Sec 3	45 min
4	Schema-First + ValidatingReader	xjc, ObjectFactory, ValidatingOrderReader, XXE	Sec 4	45 min

Quick Start

1. Unzip entities-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests
3. Run: mvn test -Dtest=SmokeTest — all 8 baseline tests must PASS.
4. Deploy: cp target/entities-lab.war $CATALINA_HOME/webapps/
5. Test: curl -H 'Accept: application/xml' http://localhost:8080/entities-lab/api/health
6. Expected: <?xml...?>
<health><status>UP</status><service>EntitiesLab</service>...</health>

Pre-built (do not modify)

HealthResource, Product, Order, OrderLine, Address, OrderStatus
ProductRepository, OrderRepository, LocalDateAdapter, CurrencyAdapter, SmokeTest

Skeletons (you implement these)

ProductCsvWriter — Lab 1 Task 1.1
ProductCsvReader — Lab 1 Task 1.2

JaxbContextResolver — Lab 2 (replaces Jackson ContextResolver)
Order, OrderLine, Address — Lab 3 (add JAXB annotations)
ValidatingOrderReader — Lab 4 Task 4.4

LAB 1

Custom Entity Provider — CSV Writer and Reader

Section: Section 1

45 min

Objectives

- Implement a complete MessageBodyWriter<Product> that produces text/csv
- Implement a matching MessageBodyReader<Product> that consumes text/csv
- Verify provider selection: CSV writer chosen over XML for Accept:text/csv
- Verify that the built-in JAXB XML provider handles Accept:application/xml

Package Namespace & Data Format — Important

All code uses javax.ws.rs.* and javax.xml.bind.* (NOT jakarta.*) — correct for Java 11 / Tomcat 9.

All resource endpoints use @Produces(APPLICATION_XML) and @Consumes(APPLICATION_XML).
Domain model classes carry @XmlRootElement so Jersey/JAXB marshals them automatically.
Lab 1 CSV provider: text/csv (unchanged — this is a custom format, not JSON/XML).

Correct imports:
import javax.ws.rs.*; import javax.ws.rs.core.*; import javax.ws.rs.ext.*;
import javax.xml.bind.*; import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.*;

Task 1.1 — Implement ProductCsvWriter

Open ProductCsvWriter.java. All imports already use javax.ws.rs.*. Implement the three interface methods:

```
import javax.ws.rs.Produces;
import javax.ws.rs.WebApplicationException;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.MessageBodyWriter;
import javax.ws.rs.ext.Provider;

@Provider
@Produces("text/csv")
public class ProductCsvWriter implements MessageBodyWriter<Product> {

    // isWriteable — return true when type is Product AND mediaType is
text/csv
    @Override
    public boolean isWriteable(Class<?> type, Type genericType,
                               Annotation[] annotations, MediaType mediaType)
{
        return Product.class.isAssignableFrom(type)
            && mediaType.isCompatible(new MediaType("text","csv"));
    }

    // getSize — always return -1 (container uses chunked encoding)
```

```
    @Override
    public long getSize(Product p, Class<?> type, Type genericType,
                        Annotation[] annotations, MediaType mediaType) {
        return -1;
    }

    // writeTo — write CSV header + one data row; NEVER close entityStream
    @Override
    public void writeTo(Product p, Class<?> type, Type genericType,
                        Annotation[] annotations, MediaType mediaType,
                        MultivaluedMap<String,Object> httpHeaders,
                        OutputStream entityStream)
            throws IOException, WebApplicationException {
        PrintWriter pw = new PrintWriter(
            new OutputStreamWriter(entityStream, StandardCharsets.UTF_8));
        pw.println("id,name,category,price");
        pw.printf("%s,%s,%s,%.2f%n",
            p.getId(), p.getName(), p.getCategory(), p.getPrice());
        pw.flush();
        // NEVER close entityStream — the container owns it
    }
}

# Test CSV writer (note: XML is the default — must request CSV explicitly):
curl -H 'Accept: text/csv' http://localhost:8080/entities-
lab/api/products/P001
# Expected:
# id,name,category,price
# P001,Clean Code,books,39.99

# XML provider still handles application/xml:
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/products/P001
# Expected: <?xml...?> <product id="P001"><name>Clean
Code</name>...</product>

# No writer for text/html:
curl -v -H 'Accept: text/html' http://localhost:8080/entities-
lab/api/products/P001
# Expected: 406 Not Acceptable
```

Write your completed ProductCsvWriter implementation:

Task 1.2 — Implement ProductCsvReader

Implement the complementary reader so that POST /api/products/csv with a CSV body creates a Product and returns XML:

```
import javax.ws.rs.BadRequestException;
import javax.ws.rs.Consumes;
import javax.ws.rs.ext.MessageBodyReader;
import javax.ws.rs.ext.Provider;

@Provider
@Consumes("text/csv")
public class ProductCsvReader implements MessageBodyReader<Product> {

    @Override
    public boolean isReadable(Class<?> type, Type genericType,
                              Annotation[] annotations, MediaType mediaType)
{
        return Product.class.isAssignableFrom(type)
            && mediaType.isCompatible(new MediaType("text","csv"));
    }

    @Override
    public Product readFrom(Class<Product> type, Type genericType,
                            Annotation[] annotations, MediaType mediaType,
                            MultivaluedMap<String,String> httpHeaders,
                            InputStream entityStream)
            throws IOException, WebApplicationException {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(entityStream, StandardCharsets.UTF_8));
        reader.readLine();              // skip header line
        String data = reader.readLine(); // data row
        if (data == null || data.isBlank())
            throw new BadRequestException("Empty CSV body");
        String[] parts = data.split(",");
        if (parts.length < 4)
            throw new BadRequestException("CSV must have 4 columns");
        Product p = new Product();
        p.setId(parts[0].trim());
        p.setName(parts[1].trim());
        p.setCategory(parts[2].trim());
        p.setPrice(new BigDecimal(parts[3].trim()));
        return p;
    }
}

# Test CSV reader — response is XML (application/xml):
curl -X POST http://localhost:8080/entities-lab/api/products/csv \
     -H 'Content-Type: text/csv' \
     -H 'Accept: application/xml' \
     -d $'id,name,category,price\nPNEW,Refactoring,books,44.99'
# Expected: 201 Created
#   Location: http://.../api/products/PNEW
#   Body: <product id="PNEW"><name>Refactoring</name>...</product>
```

Write your completed ProductCsvReader implementation:

Q1

How does the runtime choose your ProductCsvWriter over the built-in XML provider? Trace the two conditions in isWriteable() that must both be true.

Your answer:

Q2

What happens if you call entityStream.close() at the end of writeTo()? Why is this a mistake?

Your answer:

Lab 1 Checkpoint

- ✓ GET /api/products/P001 Accept:text/csv returns two-line CSV
- ✓ GET /api/products/P001 Accept:application/xml returns JAXB XML
- ✓ GET /api/products/P001 Accept:text/html returns 406
- ✓ POST /api/products/csv Content-Type:text/csv returns 201 + XML body

LAB 2

ContextResolver<JAXBContext> — Shared Context Configuration

Section: Section 2

30 min

Objectives

- Implement ContextResolver<JAXBContext> to supply a pre-built, shared JAXBContext
- Verify that LocalDate fields serialise correctly as yyyy-MM-dd strings (via LocalDateAdapter)
- Verify that BigDecimal prices appear with exactly 2 decimal places (via CurrencyAdapter)
- Understand the JAXBContext threading contract and why sharing matters

Package Namespace & Data Format — Important

All code uses javax.ws.rs.* and javax.xml.bind.* (NOT jakarta.*) — correct for Java 11 / Tomcat 9.

All resource endpoints use @Produces(APPLICATION_XML) and @Consumes(APPLICATION_XML).
Domain model classes carry @XmlRootElement so Jersey/JAXB marshals them automatically.
Lab 1 CSV provider: text/csv (unchanged — this is a custom format, not JSON/XML).

Correct imports:
import javax.ws.rs.*; import javax.ws.rs.core.*; import javax.ws.rs.ext.*;
import javax.xml.bind.*; import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.*;

Why is a ContextResolver needed here?

Without JaxbContextResolver, Jersey creates a NEW JAXBContext for each domain class it encounters. This default context:
- Does not know about LocalDateAdapter → orderDate marshals incorrectly or fails
- Does not know about CurrencyAdapter → BigDecimal marshals without fixed scale
- Creates the context on every request → performance cost

With JaxbContextResolver, one shared context is created once at startup covering all model classes. It uses the adapters registered in package-info.java and provides consistent, correctly formatted XML throughout the application.

Task 2.1 — Implement JaxbContextResolver

Open JaxbContextResolver.java. Build the shared JAXBContext in the constructor, covering all model classes:

```
import javax.ws.rs.ext.ContextResolver;
import javax.ws.rs.ext.Provider;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.JAXBException;

@Provider
```

```
public class JaxbContextResolver implements ContextResolver<JAXBContext> {

    private final JAXBContext ctx;

    public JaxbContextResolver() throws JAXBException {
        // Create one JAXBContext covering ALL model classes.
        // JAXBContext is thread-safe — create ONCE, reuse forever.
        // Listing Order.class alone would suffice (it references Address,
        // OrderLine etc.) but listing all classes is safer and clearer.
        ctx = JAXBContext.newInstance(
            Order.class, OrderLine.class, Address.class,
            OrderStatus.class, Product.class);
    }

    @Override
    public JAXBContext getContext(Class<?> type) {
        // Return the shared context for every type.
        // Return null to let Jersey use its default for specific types.
        return ctx;
    }
}
```

Task 2.2 — Verify Adapter Behaviour

Test the effects of the registered adapters. Before implementing Lab 3 annotations, use the order endpoint and observe field format:

```
# 1. LocalDate → yyyy-MM-dd (via LocalDateAdapter)
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/orders/ORD-001
# After Lab 3 annotations: <orderDate>2024-06-15</orderDate>
# Without adapter: LocalDate cannot be marshalled (XMLGregorianCalendar
expected)

# 2. BigDecimal → 2 decimal places (via CurrencyAdapter)
# Expected: <totalAmount>99.97</totalAmount>  (not 99.970000000 etc.)

# 3. internalRef must NOT appear in XML (via @XmlTransient in Lab 3)
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/orders/ORD-001
# Confirm: internalRef is absent from output

# 4. Product XML output:
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/products/P001
# Expected: <product id="P001">
#             <name>Clean Code</name>
#             <category>books</category>
#             <price>39.99</price>
#             <stockLevel>50</stockLevel>
#             <active>true</active>
#           </product>
```

Paste the XML output for GET /api/products/P001 after implementing the resolver:

Task 2.3 — Threading Proof

Verify the threading contract by adding a log line in the constructor and making several concurrent requests:

```
// Add this line to the constructor to prove it runs only ONCE:
System.out.println("[JaxbContextResolver] Creating JAXBContext — "+
    Thread.currentThread().getName());

// Deploy and make 5 rapid requests:
for i in 1 2 3 4 5; do
  curl -s -H 'Accept: application/xml' \
       http://localhost:8080/entities-lab/api/products/P001 > /dev/null &
done
wait

# Check Tomcat log — the constructor message should appear ONLY ONCE
grep 'JaxbContextResolver' $CATALINA_HOME/logs/catalina.out | wc -l
# Expected: 1
```

Q1

JAXBContext is thread-safe but Marshaller and Unmarshaller are NOT. What does this mean in practice for a Jersey resource method that marshals an Order?

Your answer:

Q2

What does getContext(Class<?> type) return if you want Jersey to use its default context for Product but your shared context for Order? Write the modified method.

Your modified getContext() + explanation:

Lab 2 Checkpoint

- ✓ mvn test -Dtest=SmokeTest all 8 tests still PASS after adding JaxbContextResolver
- ✓ GET /api/products/P001 returns well-formed XML with @XmlRootElement structure
- ✓ Constructor log message appears exactly ONCE in catalina.out
- ✓ LocalDate fields marshal as yyyy-MM-dd strings (confirmed in Lab 3)

LAB 3

JAXB Annotations — XML Output

Section: Section 3

45 min

Objectives

- Annotate Order with @XmlRootElement, @XmlAttribute, @XmlElementWrapper and @XmlTransient
- Annotate OrderLine and Address with their JAXB annotations
- Verify the full target XML structure is produced correctly
- Apply a package-level namespace via package-info.java

Task 3.1 — Annotate the Order Class

Open Order.java. Add all required JAXB annotations. The target XML structure is:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<order xmlns="http://example.com/shop/orders"
       orderId="ORD-001" status="PROCESSING">
  <customerName>Jane Smith</customerName>
  <customerEmail>jane@example.com</customerEmail>
  <orderDate>2024-06-15</orderDate>
  <deliveryAddress>
    <street>10 Downing Street</street>
    <city>London</city>
    <postCode>SW1A 2AA</postCode>
    <country>GB</country>
  </deliveryAddress>
  <lines>
    <line sku="BK-001">
      <productName>Clean Code</productName>
      <quantity>2</quantity>
      <unitPrice>29.99</unitPrice>
    </line>
    <line sku="BK-002">
      <productName>Effective Java</productName>
      <quantity>1</quantity>
      <unitPrice>39.99</unitPrice>
    </line>
  </lines>
  <totalAmount>99.97</totalAmount>
  <!-- internalRef must NOT appear -->
</order>
```

Annotations to add

Order class level:

```
@XmlRootElement(name="order", namespace="http://example.com/shop/orders")
@XmlType(propOrder={"customerName","customerEmail","orderDate",
                    "deliveryAddress","lines","totalAmount","notes"})
@XmlAccessorType(XmlAccessType.FIELD)
```

Field level:

- orderId → @XmlAttribute(name="orderId", required=true)
- status → @XmlAttribute(name="status")
- orderDate → @XmlElement(required=true)

```
                @XmlJavaTypeAdapter(LocalDateAdapter.class)
  lines         → @XmlElementWrapper(name="lines")
                @XmlElement(name="line")
  totalAmount   → @XmlElement(required=true)
                @XmlJavaTypeAdapter(CurrencyAdapter.class)
  internalRef   → @XmlTransient

Remember: import javax.xml.bind.annotation.*;
          import javax.xml.bind.annotation.adapters.XmlJavaTypeAdapter;
```

Write your fully-annotated Order.java:

Task 3.2 — Annotate OrderLine and Address

```
// OrderLine — sku is an XML ATTRIBUTE, not a child element:
import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.XmlJavaTypeAdapter;

@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = { "productName", "quantity", "unitPrice" })
public class OrderLine {
    @XmlAttribute(name="sku", required=true)
    private String sku;

    @XmlElement(required=true)
```

```
    private String productName;

    @XmlElement(required=true)
    private BigInteger quantity;

    @XmlElement(required=true)
    @XmlJavaTypeAdapter(CurrencyAdapter.class)
    private BigDecimal unitPrice;
    // ... constructors and getters unchanged
}

// Address — all fields are child elements:
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = { "street", "city", "postCode", "country" })
public class Address {
    // No @XmlAttribute — all fields become child elements
    private String street;
    private String city;
    private String postCode;
    private String country;
    // ... constructors and getters unchanged
}
```

Task 3.3 — Test the XML Output

```
# Get a single order as XML:
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/orders/ORD-001

# Verify ALL of the following in the output:
# ✓  orderId="ORD-001" and status="PROCESSING" are XML ATTRIBUTES
# ✓  <orderDate>2024-06-15</orderDate>  (yyyy-MM-dd, NOT an array)
# ✓  <lines><line sku="BK-001">...</line></lines>  (wrapper + sku attribute)
# ✓  <totalAmount>99.97</totalAmount>  (exactly 2 decimal places)
# ✓  internalRef does NOT appear anywhere
# ✓  namespace xmlns="http://example.com/shop/orders" on root element

# POST an order as XML (now that Order is annotated for unmarshalling too):
curl -v -X POST http://localhost:8080/entities-lab/api/orders \
     -H 'Content-Type: application/xml' \
     -H 'Accept: application/xml' \
     -d '<?xml version="1.0"?>
         <order xmlns="http://example.com/shop/orders"
                orderId="ORD-999" status="NEW">
           <customerName>Test Customer</customerName>
           <customerEmail>test@example.com</customerEmail>
           <totalAmount>9.99</totalAmount>
         </order>'
# Expected: 201 Created + Location + XML body of saved order
```

Paste the XML output for GET /api/orders/ORD-001 and confirm each checkpoint:

Task 3.4 — Namespace via package-info.java

Create src/main/java/com/example/model/package-info.java to apply the namespace to all model classes via package-level registration:

```
@XmlSchema(
    namespace = "http://example.com/shop/orders",
    elementFormDefault = XmlNsForm.QUALIFIED
)
package com.example.model;

import javax.xml.bind.annotation.XmlNsForm;
import javax.xml.bind.annotation.XmlSchema;

// After adding this file, remove the namespace= attribute from
// @XmlRootElement on Order.java:
//   @XmlRootElement(name="order")   ← namespace now comes from package-info

# Confirm namespace is still present in XML output after the change:
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/orders/ORD-001
# xmlns="http://example.com/shop/orders" must still appear on <order>
```

Q1

Without @XmlElementWrapper on the lines field, what would the XML output look like? Why is the wrapper element important for API consumers?

Your answer:

Lab 3 Checkpoint

- ✓ GET /api/orders/ORD-001 Accept:application/xml returns namespace-qualified XML
- ✓ orderId and status appear as XML attributes (not child elements)
- ✓ <lines><line sku="...">...</line></lines> structure is correct
- ✓ <orderDate>2024-06-15</orderDate> (yyyy-MM-dd string, not array)
- ✓ internalRef is absent from XML output
- ✓ POST XML order returns 201 + Location + XML body

LAB 4 **Schema-First with xjc + ValidatingOrderReader** 45 min
Section: Section 4

Objectives

- Write an XSD for a Catalogue domain and generate Java binding classes with xjc
- Use ObjectFactory to create instances of generated classes in a resource method
- Implement ValidatingOrderReader to enforce order-schema.xsd on inbound POST requests
- Configure XXE prevention on all XMLInputFactory instances
- Observe schema validation errors returned as 400 Bad Request

Task 4.1 — Write the Catalogue XSD

Create src/main/xsd/catalogue-schema.xsd. The schema must model a book catalogue:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns:tns="http://example.com/library"
    targetNamespace="http://example.com/library"
    elementFormDefault="qualified">

  <!-- Simple type: ISBN pattern -->
  <xs:simpleType name="IsbnType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[0-9]{3}-[0-9]{10}"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Simple type: Genre enumeration -->
  <xs:simpleType name="GenreType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="FICTION"/>
      <xs:enumeration value="NON_FICTION"/>
      <xs:enumeration value="SCIENCE"/>
      <xs:enumeration value="BIOGRAPHY"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Complex type: Book -->
  <xs:complexType name="BookType">
    <xs:sequence>
      <xs:element name="title"         type="xs:string"       minOccurs="1"/>
      <xs:element name="isbn"          type="tns:IsbnType"/>
      <xs:element name="genre"         type="tns:GenreType"/>
      <xs:element name="publishedYear" type="xs:gYear"/>
      <xs:element name="pages"         type="xs:positiveInteger"
minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="language" type="xs:string" use="required"/>
  </xs:complexType>

  <!-- Complex type: Catalogue -->
  <xs:complexType name="CatalogueType">
    <xs:sequence>
      <xs:element name="name"    type="xs:string"/>
      <xs:element name="created" type="xs:date"/>
      <xs:element name="book"    type="tns:BookType"
```

```
                  minOccurs="0"  maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Global root element -->
  <xs:element name="catalogue" type="tns:CatalogueType"/>

</xs:schema>
```

Task 4.2 — Generate Binding Classes with xjc

Uncomment the jaxb2-maven-plugin block in pom.xml and fill in the packageName. Then run:

```
<!-- In pom.xml — uncomment and configure the plugin: -->
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>jaxb2-maven-plugin</artifactId>
    <version>2.5.0</version>
    <executions>
        <execution><goals><goal>xjc</goal></goals></execution>
    </executions>
    <configuration>
        <schemaDirectory>src/main/xsd</schemaDirectory>
        <schemaFiles>catalogue-schema.xsd</schemaFiles>
        <bindingIncludes>
            <bindingInclude>NONE</bindingInclude>
        </bindingIncludes>
        <packageName>com.example.library.generated</packageName>
        <outputDirectory>
            ${project.build.directory}/generated-sources/jaxb
        </outputDirectory>
        <clearOutputDir>false</clearOutputDir>
    </configuration>
</plugin>

# Run xjc:
mvn generate-sources

# List generated files:
find target/generated-sources/jaxb -name '*.java' | sort
# Expected:
#   BookType.java
#   CatalogueType.java
#   GenreType.java
#   ObjectFactory.java
#   package-info.java

# Mark target/generated-sources/jaxb as Generated Sources Root in your IDE.
```

Q1

List every .java file generated by xjc. What is the purpose of ObjectFactory.java and why should you use it rather than calling 'new CatalogueType()' directly?

Your answer:

Task 4.3 — Use ObjectFactory in CatalogueResource

Implement GET /api/catalogue using the generated classes. Always use ObjectFactory to create instances:

```
import com.example.library.generated.*;
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.xml.bind.JAXBElement;

@Path("/catalogue")
@Produces(MediaType.APPLICATION_XML)
public class CatalogueResource {

    @GET
    public Response get() {
        ObjectFactory factory = new ObjectFactory();

        // Create a book using ObjectFactory (never new BookType())
        BookType book = factory.createBookType();
        book.setTitle("Clean Code");
        book.setIsbn("978-0132350884");
        book.setGenre(GenreType.NON_FICTION);
        book.setLanguage("EN");

        // Create the catalogue
        CatalogueType cat = factory.createCatalogueType();
        cat.setName("Tech Reads");
        cat.getBook().add(book);

        // Wrap as JAXBElement — required for the global root element
declaration
        JAXBElement<CatalogueType> elem = factory.createCatalogue(cat);
        return Response.ok(elem).build();
    }
}

# Test:
curl -H 'Accept: application/xml' http://localhost:8080/entities-
lab/api/catalogue
# Expected: <?xml...?>
#   <catalogue xmlns="http://example.com/library">
#     <name>Tech Reads</name>
#     <book language="EN">
#       <title>Clean Code</title>
#       <isbn>978-0132350884</isbn>
#       <genre>NON_FICTION</genre>
#     </book>
#   </catalogue>
```

Write your completed CatalogueResource:

Q2

Why must you use factory.createCatalogue(cat) rather than returning CatalogueType directly? What happens if you return CatalogueType without the JAXBElement wrapper?

Your answer:

Task 4.4 — Implement ValidatingOrderReader with XXE Prevention

Open ValidatingOrderReader.java. Implement the static initialiser, readFrom(), and buildSafeSource(). Pay special attention to the XXE prevention properties:

```
import javax.ws.rs.BadRequestException;
import javax.ws.rs.Consumes;
import javax.ws.rs.ext.MessageBodyReader;
import javax.ws.rs.ext.Provider;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Unmarshaller;
import javax.xml.bind.ValidationEvent;
import javax.xml.stream.XMLInputFactory;
import javax.xml.stream.XMLStreamException;
import javax.xml.transform.Source;
import javax.xml.transform.stax.StAXSource;
import javax.xml.validation.Schema;
import javax.xml.validation.SchemaFactory;
import javax.xml.XMLConstants;

@Provider
@Consumes(MediaType.APPLICATION_XML)
public class ValidatingOrderReader implements MessageBodyReader<Order> {

    // Thread-safe singletons — create ONCE
    private static final JAXBContext CTX;
    private static final Schema      SCHEMA;

    static {
        try {
            CTX = JAXBContext.newInstance(Order.class);

            SchemaFactory sf = SchemaFactory.newInstance(
```

```
                XMLConstants.W3C_XML_SCHEMA_NS_URI);
            SCHEMA = sf.newSchema(
                ValidatingOrderReader.class
                    .getResource("/xsd/order-schema.xsd"));
        } catch (Exception e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    @Override
    public boolean isReadable(Class<?> type, Type genericType,
                              Annotation[] annotations, MediaType mediaType)
{
        return Order.class.isAssignableFrom(type)
            && mediaType.isCompatible(MediaType.APPLICATION_XML_TYPE);
    }

    @Override
    public Order readFrom(Class<Order> type, Type genericType,
                          Annotation[] annotations, MediaType mediaType,
                          MultivaluedMap<String,String> httpHeaders,
                          InputStream entityStream)
            throws IOException {
        try {
            Source safeSource = buildSafeSource(entityStream);
            Unmarshaller u = CTX.createUnmarshaller();
            u.setSchema(SCHEMA);
            u.setEventHandler(event -> {
                if (event.getSeverity() >= ValidationEvent.ERROR)
                    throw new RuntimeException(
                        "Schema violation: " + event.getMessage());
                return true;
            });
            return (Order) u.unmarshal(safeSource);
        } catch (Exception e) {
            throw new BadRequestException(
                "Invalid XML: " + e.getMessage(), e);
        }
    }

    private Source buildSafeSource(InputStream is)
            throws IOException, XMLStreamException {
        XMLInputFactory xif = XMLInputFactory.newInstance();
        // SECURITY — both properties are mandatory for XXE prevention:
        xif.setProperty(
            XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
        xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);
        return new StAXSource(xif.createXMLStreamReader(is));
    }
}

# Test — valid XML (order-schema.xsd passes):
curl -v -X POST http://localhost:8080/entities-lab/api/orders \
     -H 'Content-Type: application/xml' \
     -H 'Accept: application/xml' \
     -d @src/test/resources/xml/valid-order.xml
# Expected: 201 Created

# Test — missing required orderId attribute → 400:
curl -v -X POST http://localhost:8080/entities-lab/api/orders \
     -H 'Content-Type: application/xml' \
     -d '<?xml version="1.0"?>
         <order xmlns="http://example.com/shop/orders" status="NEW">
           <customerName>Missing ID</customerName>
```

```
        </order>'
# Expected: 400 Bad Request
```

Write your completed ValidatingOrderReader (static block + readFrom() + buildSafeSource()):

Q3 — Security

What is an XXE (XML External Entity) attack? Write an example DOCTYPE declaration that attempts to read /etc/passwd.
Which two XMLInputFactory properties prevent it, and why must BOTH be set?

Your answer:

Lab 4 Checkpoint

✓ mvn generate-sources produces CatalogueType, BookType, GenreType, ObjectFactory
✓ GET /api/catalogue returns namespace-qualified XML catalogue
✓ POST /api/orders with valid-order.xml returns 201
✓ POST with XML missing orderId attribute returns 400 Bad Request
✓ buildSafeSource() sets IS_SUPPORTING_EXTERNAL_ENTITIES=false AND SUPPORT_DTD=false

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 1 — ProductCsvWriter.writeTo()

```
public void writeTo(Product p, Class<?> type, Type genericType,
        Annotation[] annotations, MediaType mediaType,
        MultivaluedMap<String,Object> httpHeaders,
        OutputStream entityStream) throws IOException {
    PrintWriter pw = new PrintWriter(
        new OutputStreamWriter(entityStream, StandardCharsets.UTF_8));
    pw.println("id,name,category,price");
    pw.printf("%s,%s,%s,%.2f%n",
        p.getId(), p.getName(), p.getCategory(), p.getPrice());
    pw.flush();
    // Do NOT close entityStream
}
```

Lab 2 — JaxbContextResolver

```
import javax.ws.rs.ext.ContextResolver;
import javax.ws.rs.ext.Provider;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.JAXBException;

@Provider
public class JaxbContextResolver implements ContextResolver<JAXBContext> {
    private final JAXBContext ctx;
    public JaxbContextResolver() throws JAXBException {
        ctx = JAXBContext.newInstance(
            Order.class, OrderLine.class, Address.class,
            OrderStatus.class, Product.class);
    }
    @Override
    public JAXBContext getContext(Class<?> type) { return ctx; }
}
```

Lab 3 — Order class annotations (abridged)

```
import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.XmlJavaTypeAdapter;

@XmlRootElement(name="order", namespace="http://example.com/shop/orders")
@XmlType(propOrder={"customerName","customerEmail","orderDate",
                    "deliveryAddress","lines","totalAmount","notes"})
@XmlAccessorType(XmlAccessType.FIELD)
public class Order {
    @XmlAttribute(name="orderId", required=true) private String orderId;
    @XmlAttribute(name="status")  private OrderStatus status;
    @XmlElement(required=true)    private String customerName;
    @XmlElement(required=true)
    @XmlJavaTypeAdapter(LocalDateAdapter.class) private LocalDate orderDate;
    @XmlElementWrapper(name="lines")
```

```
    @XmlElement(name="line")  private List<OrderLine> lines = new
ArrayList<>();
    @XmlElement(required=true)
    @XmlJavaTypeAdapter(CurrencyAdapter.class) private BigDecimal
totalAmount;
    @XmlTransient private String internalRef;
}
```

Lab 4 — buildSafeSource() for XXE prevention

```
private Source buildSafeSource(InputStream is)
        throws IOException, XMLStreamException {
    XMLInputFactory xif = XMLInputFactory.newInstance();
    // Both properties are mandatory — setting only one leaves an attack
surface
    xif.setProperty(
        XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
    xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);
    return new StAXSource(xif.createXMLStreamReader(is));
}
```